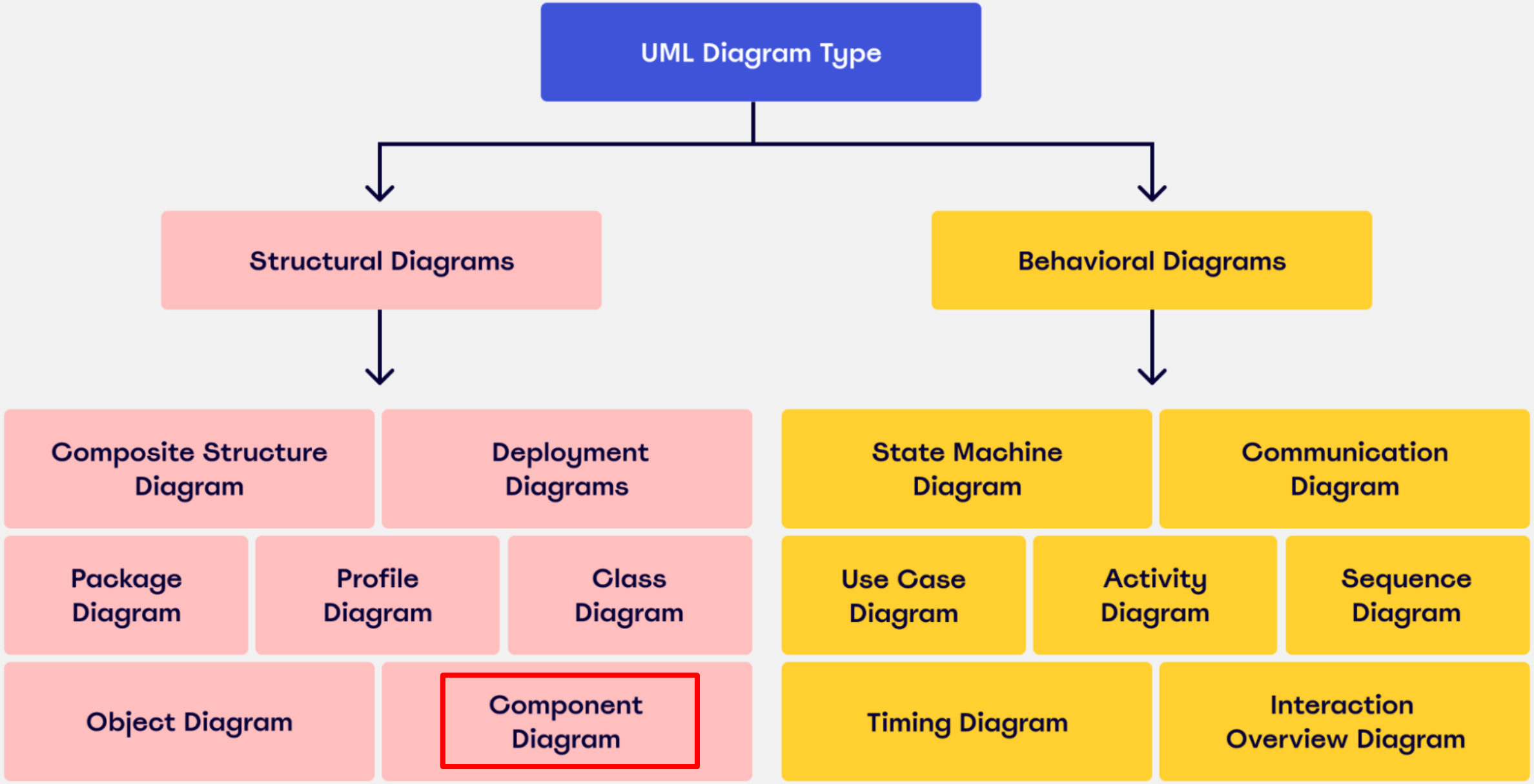
Object Oriented Analysis and Design

Component Diagram


UML Diagram Type
Structural Diagrams
Behavioral Diagrams
Composite Structure Diagram
Deployment Diagrams
Package Diagram
Profile Diagram
Class Diagram
Object Diagram
Component Diagram
State Machine Diagram
Communication Diagram
Use Case Diagram
Activity Diagram
Sequence Diagram
Timing Diagram
Interaction Overview Diagram

Why These Diagrams Matter?

- Design a software that:
  - Reads chromosomes from a dataset and creates corresponding objects, e.g., Chromosome, SNP, etc.
  - Computes population statistics, e.g., we have a number of classes that analyses chromosomes, SNPs with respect to certain criteria associated with the average population
  - Presents the results of the computation either in a Web browser or in a mobile application.

Think of it as a blueprint for your software infrastructure

Component Diagrams - Overview

- Component diagrams show the logical architecture of a system – how software is organized into modular, replaceable parts.
- **Key Characteristics of Components:**

  Encapsulated: Internal details are hidden

  Replaceable: Can swap implementations

  Reusable: Can be used in different systems

  Interface-based: Clear contracts for interaction

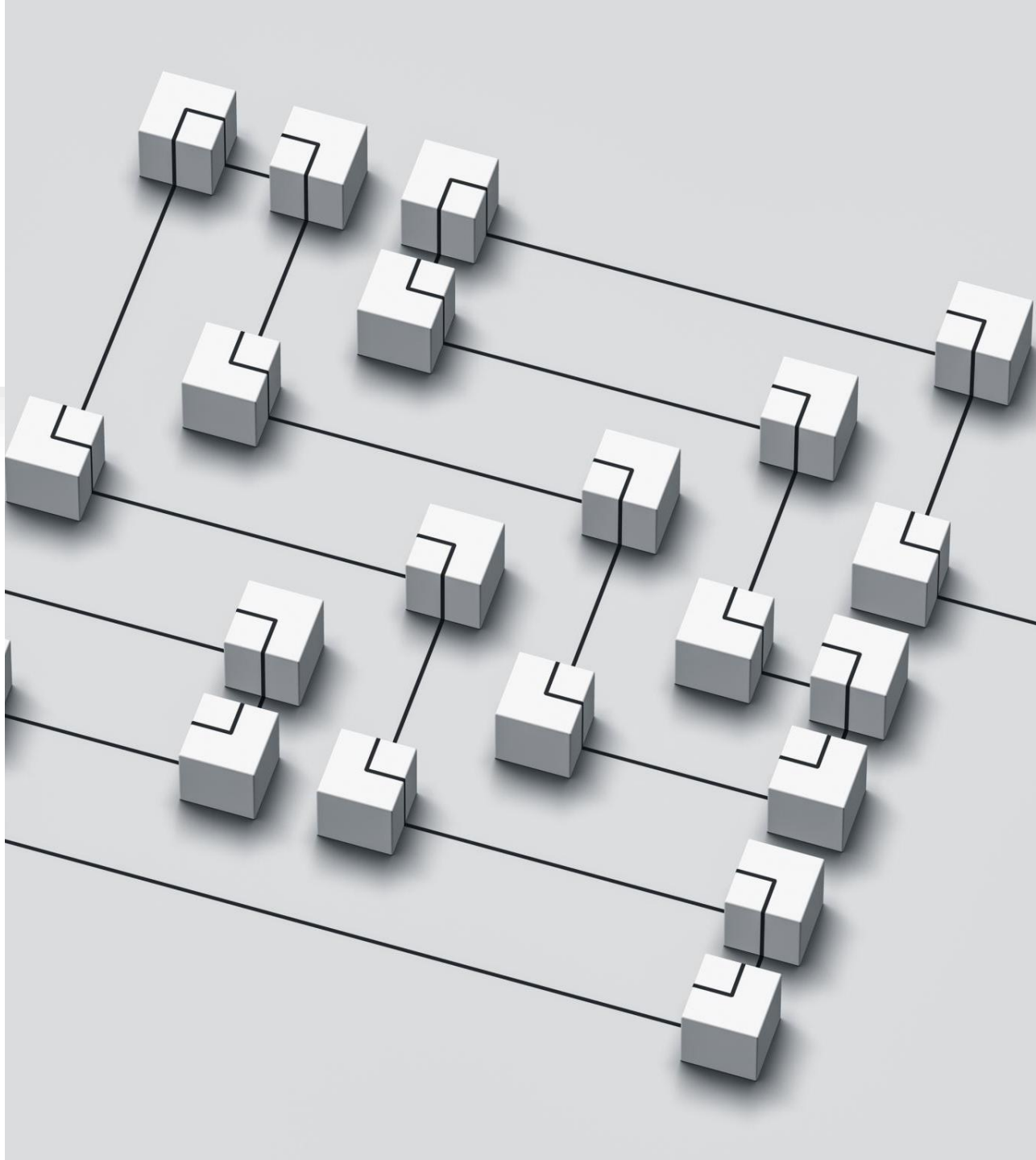
Real-World Analogy:

Like a home entertainment system - you can replace the DVD player with a Blu-ray player as long as it has the same HDMI interface. The TV doesn't need to know the internal workings of the player.

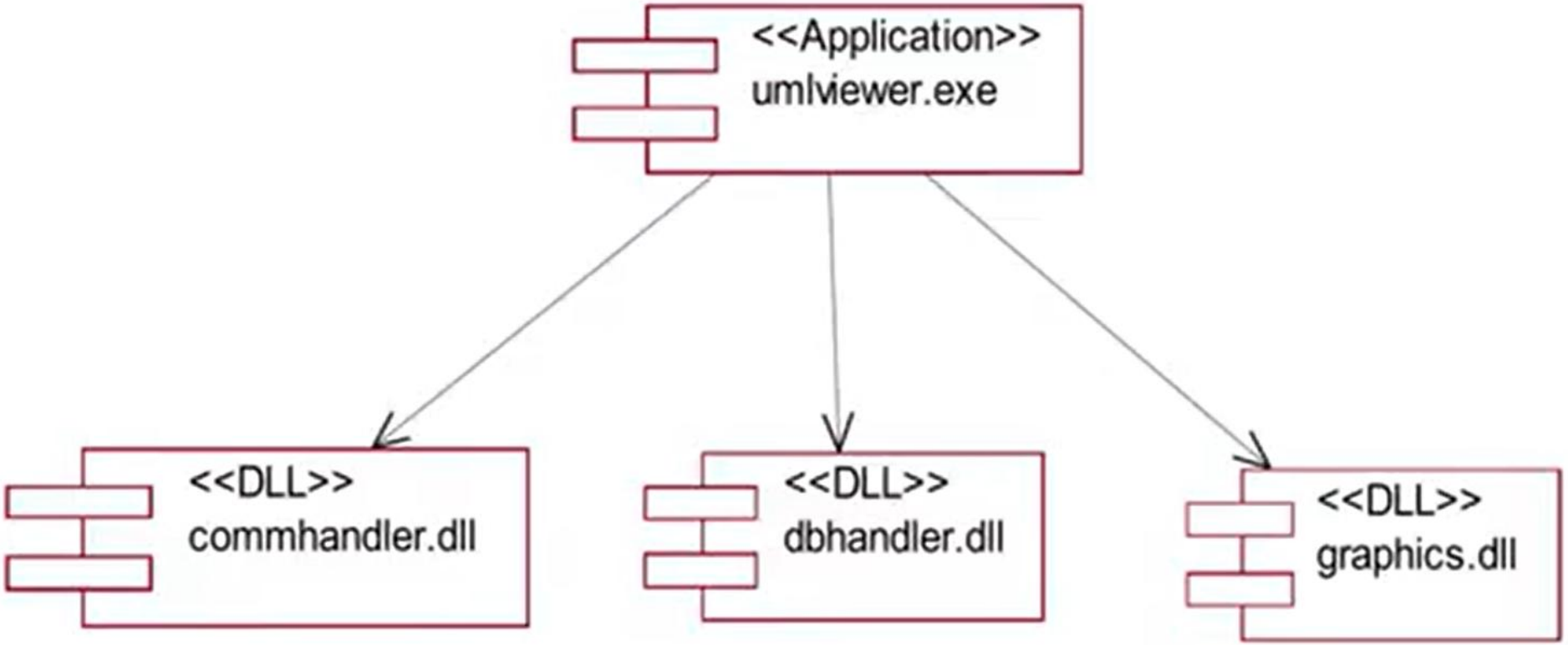
What is a UML component diagram

- The purpose of a component diagram is to show the relationship between different components in a system.

- The term component refers to a module of classes that represent independent systems or subsystems with the ability to interface with the rest of the system.



Component Diagrams



Benefits of Component Diagram

- Though component diagrams may seem complex at first glance, they are invaluable when it comes to building your system. Component diagrams can help your team:
  - Imagine the system’s physical structure.
  - Pay attention to the system’s components and how they relate.
  - Emphasise the service behaviour as it relates to the interface.

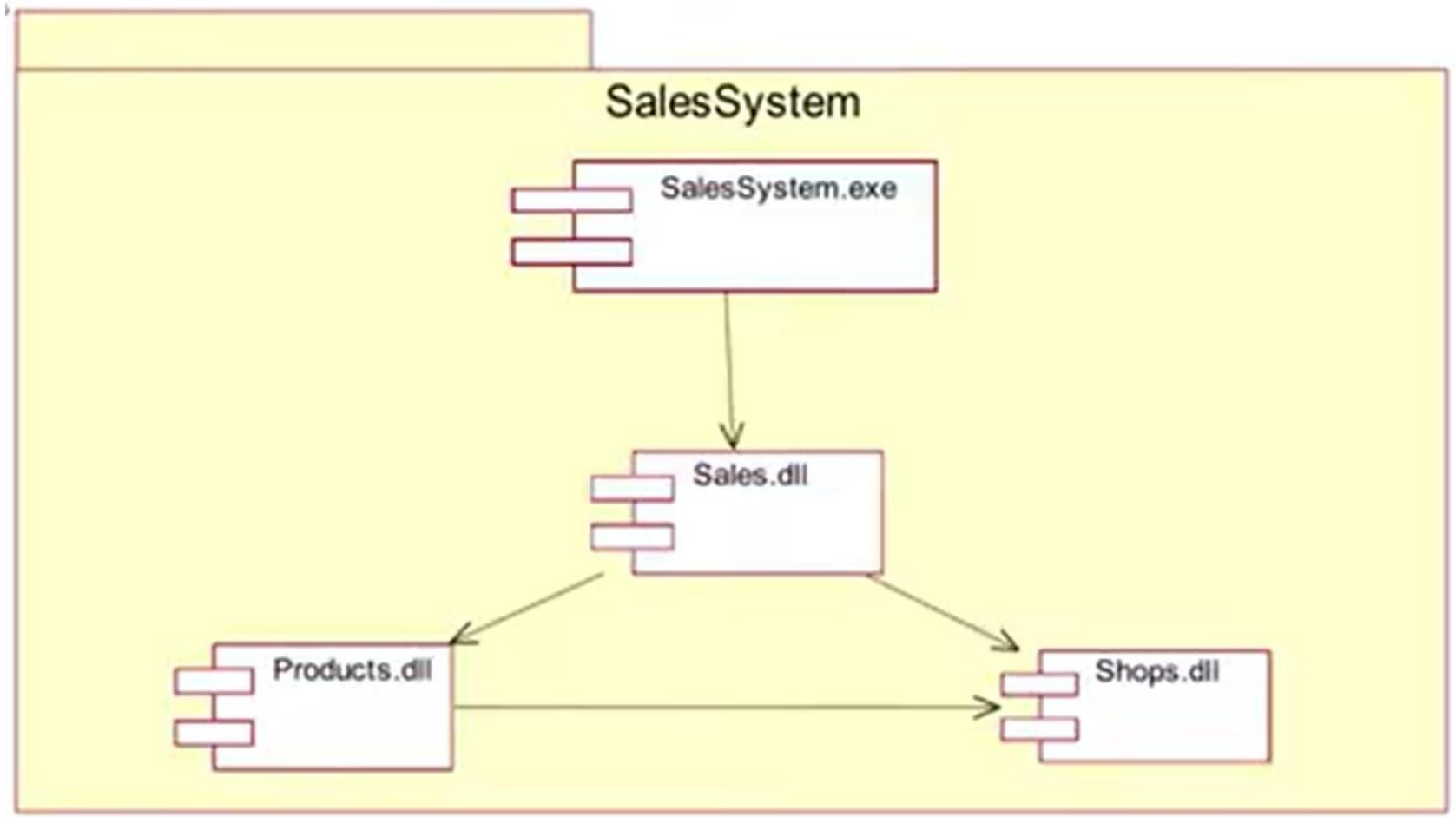
How to use a component’s diagram

- A component diagram in UML gives a bird’s-eye view of your software system.

- Understanding the exact service behaviour that each piece of your software provides will make you a better developer.

- Component diagrams can describe software systems that are implemented in any programming language or style.

Component Diagrams



Component

- A *component* is a **class** representing a modular part of a system with encapsulated content and whose **manifestation** is replaceable within its environment.

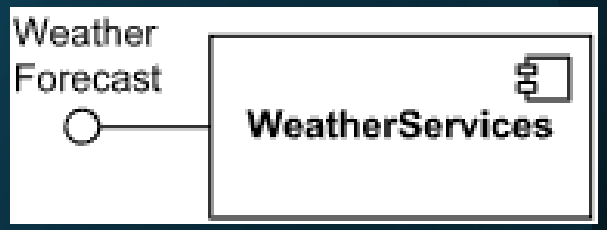
- A component is shown as a classifier rectangle with the keyword «*component*».

- If the icon symbol is shown, the keyword «component» may be omitted.

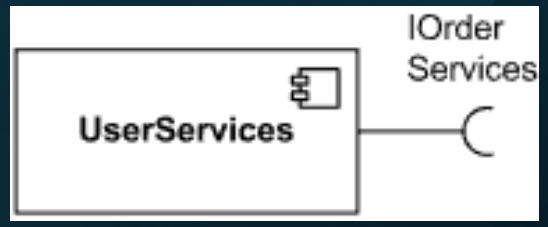




Interfaces: Provided and Required



Weather Services component provides (implements) Weather Forecast interface



User Services component requires OrderServices interface

- **Provided Interface**
- What the component offers to others
- Symbol: ⊂ (lollipop)
- The services this component implements and makes available

Required Interface

What the component needs to work

Symbol: ⊃ (socket)

The dependencies this component has on other services

Example:

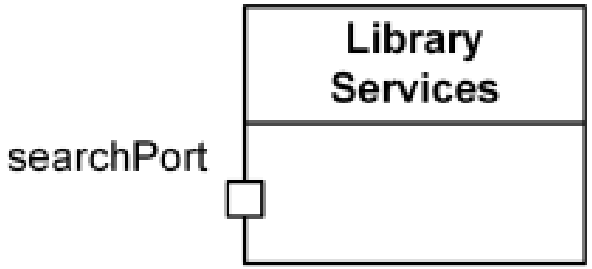
OrderService Component

- Provides: OrderManagement (createOrder, cancelOrder)
- Requires: PaymentGateway, InventoryService

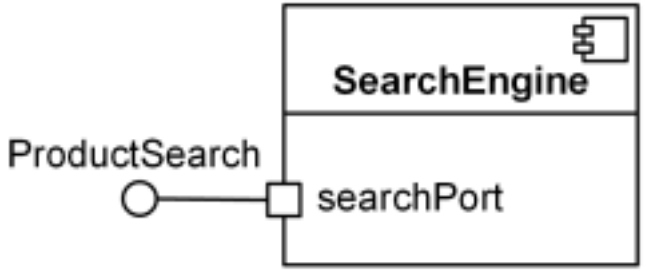
This is the foundation of Dependency Injection and loose coupling!

Ports

- Ports → Interaction points on a component (shown as small squares)
  - Simple Port: Single Interface (provided or required)
  - Complex Port: Multiple Interfaces
  - Service Port: External-facing (default)
  - Behavior Port: Connects to Component's behavior

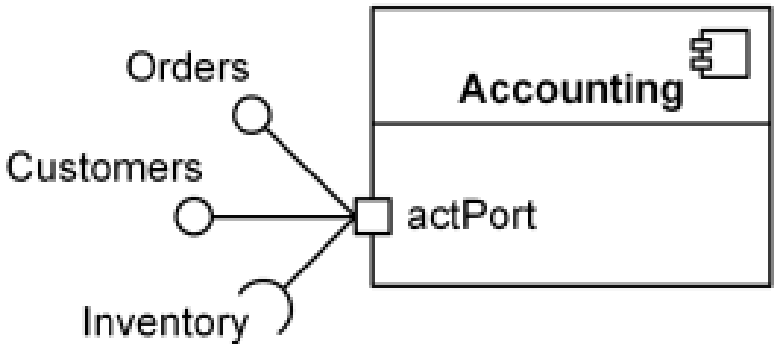


Library Services class has port searchPort.

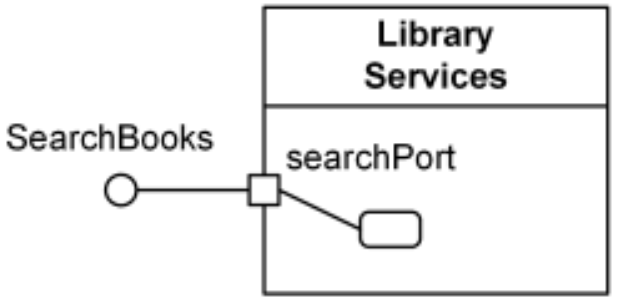


SearchEngine component has simple port searchPort with single provided ProductSearch interface.

Ports



Port actPort is by default a service port providing Orders and Customers interfaces and requiring Inventory interface.

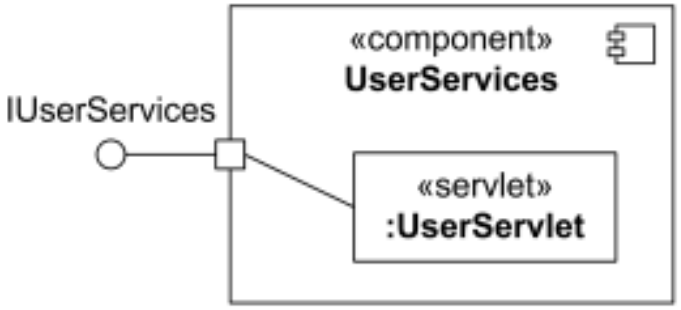


Port searchPort is behavior port with SearchBooks provided interface.

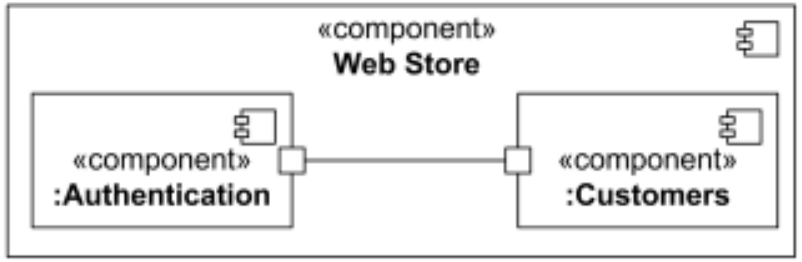
Connectors

Practical Use: Ports make it explicit WHERE connections happen - essential for microservices architectures!

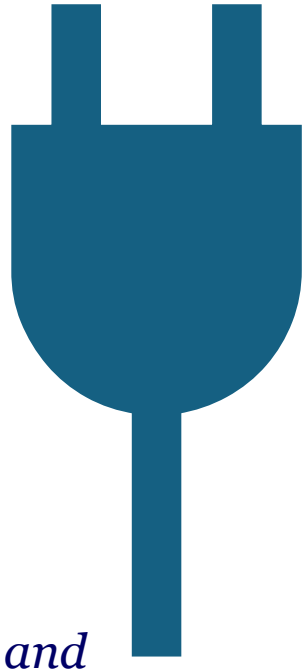
- Wires that connect components together
  - Assembly Connector: Connects provider to consumer
  - Delegation Connector: Forwards port to internal part



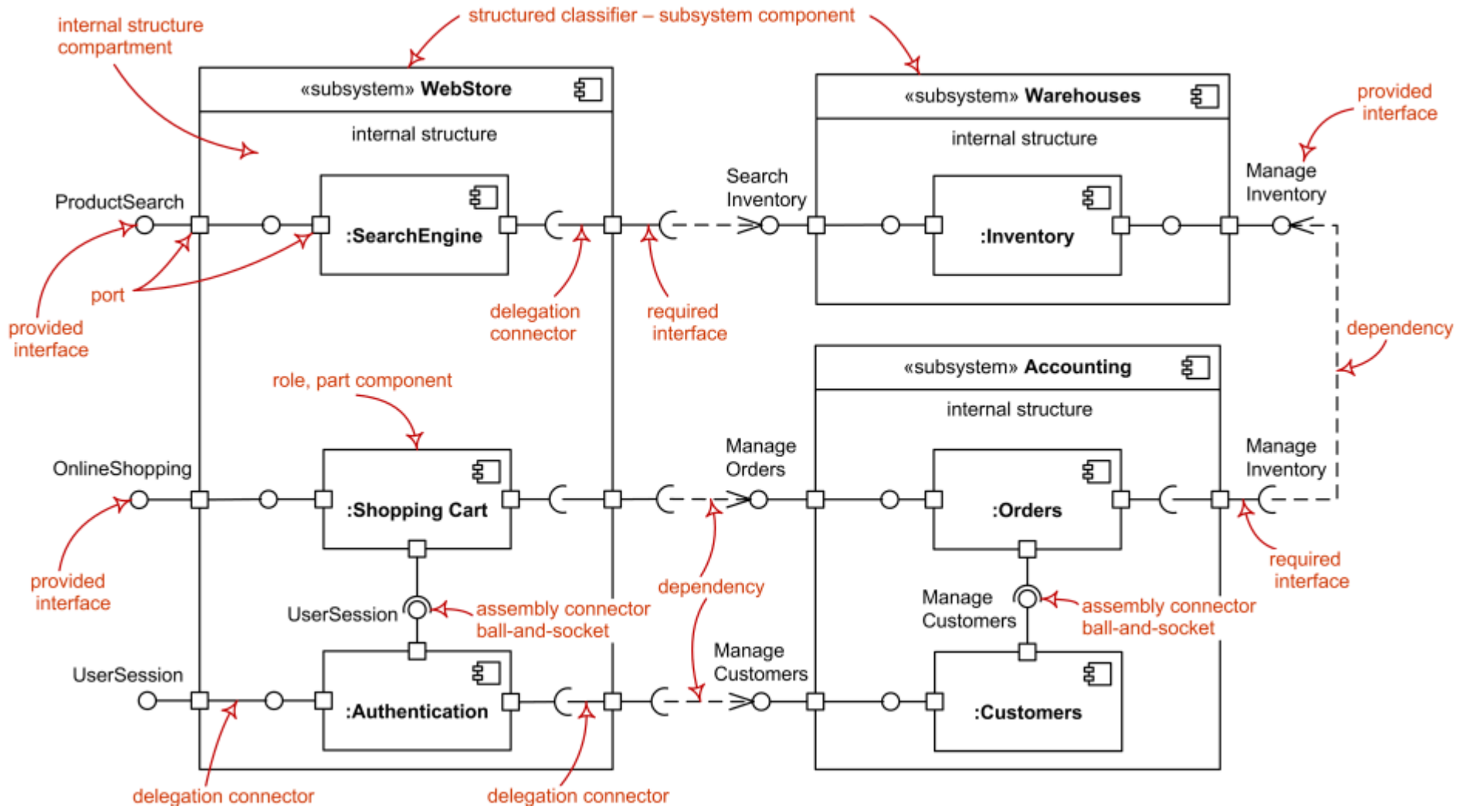
Delegation connector from the delegating port to the UserServlet part



Assembly connector between ports of Authentication and Customers components

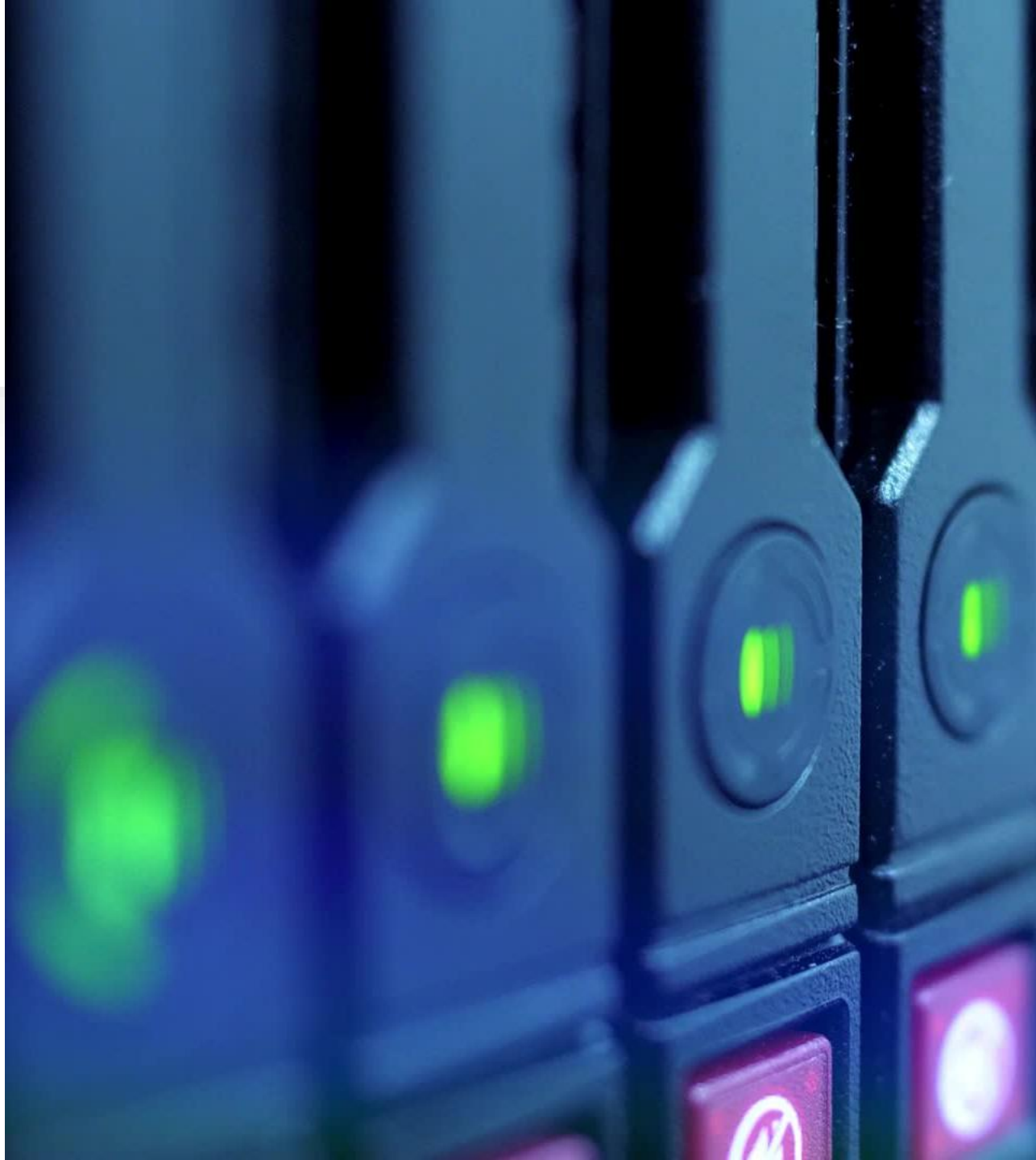


Component Diagram



Component Diagrams

For example, at a large-grained level, a SQL database engine can be modeled as a component; any database that understands the same version of SQL and supports the same transaction semantics can be substituted.

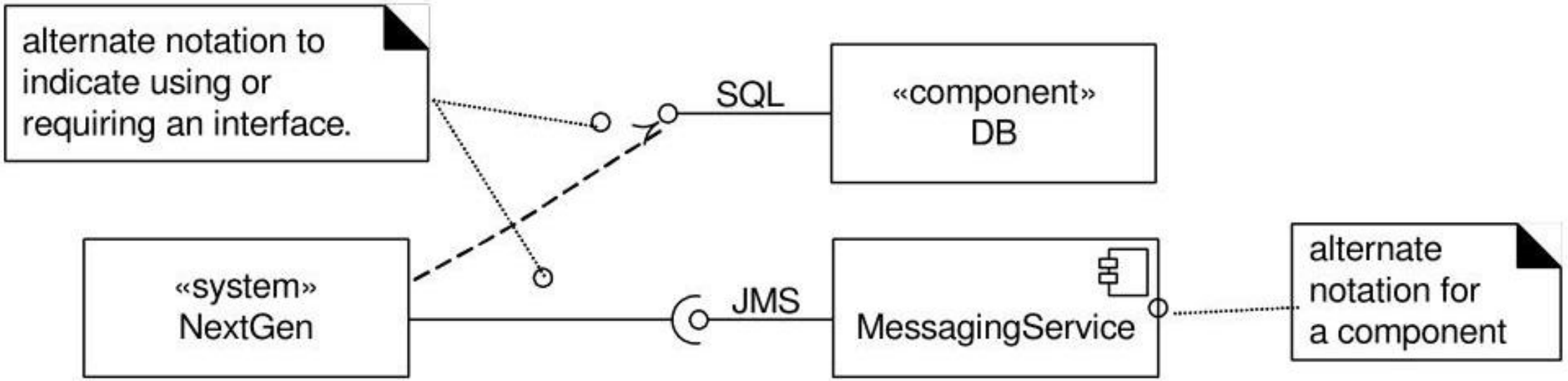


Component Diagrams

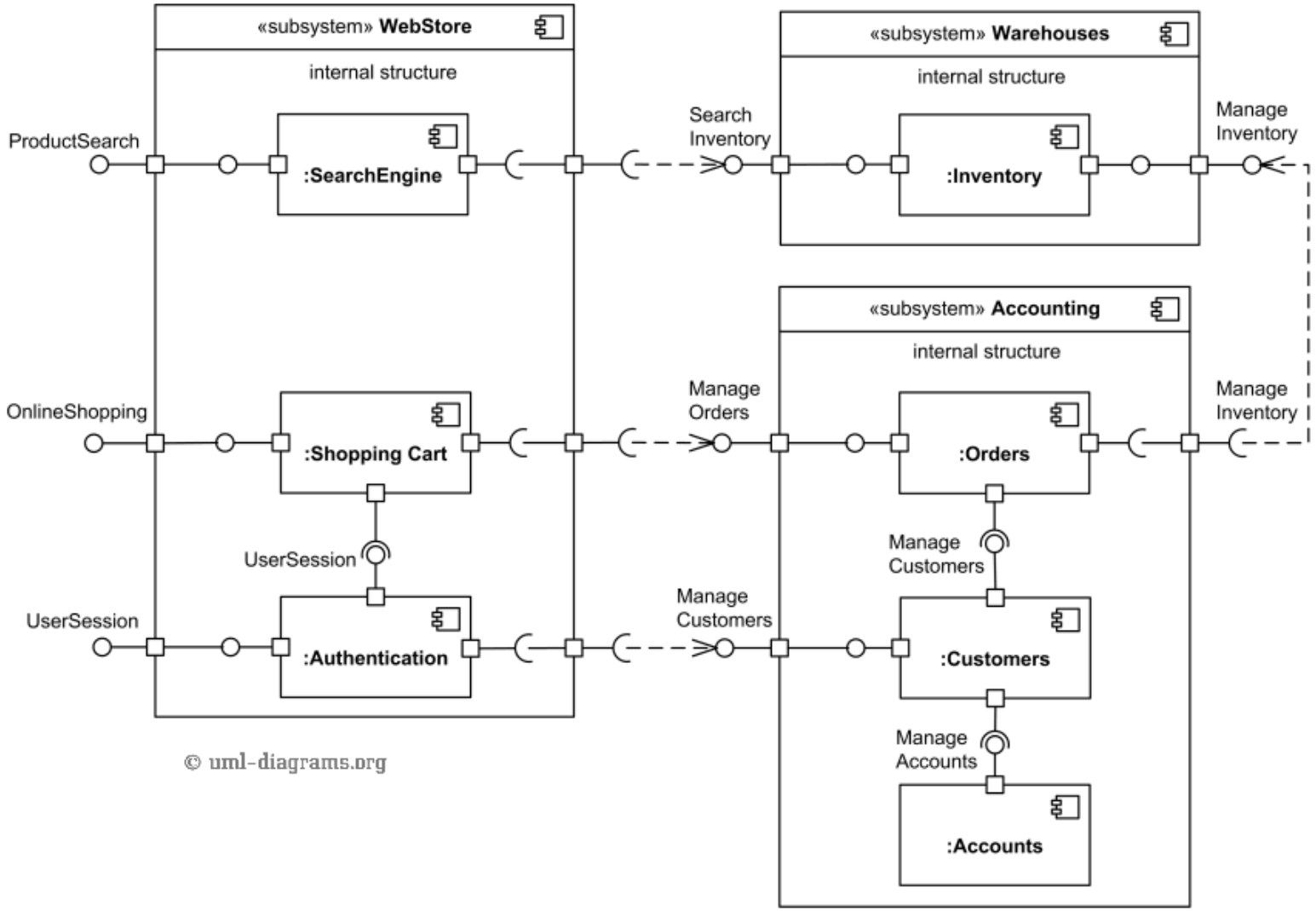
- At a finer level, any solution that implements the standard Java message service (JMS) API can be used or replaced in a system
    - JMS API is a messaging standard that allows application components based on the Java 2 platform enterprise edition (J2EE) to create, send, receive, and read messages
    - It enables distributed communication that is loosely coupled, reliable, and asynchronous.



Component Diagrams



Component Diagram Example



Case Study: E-Commerce Platform

ShopEasy - Online Shopping Platform
A modern microservices-based e-commerce system

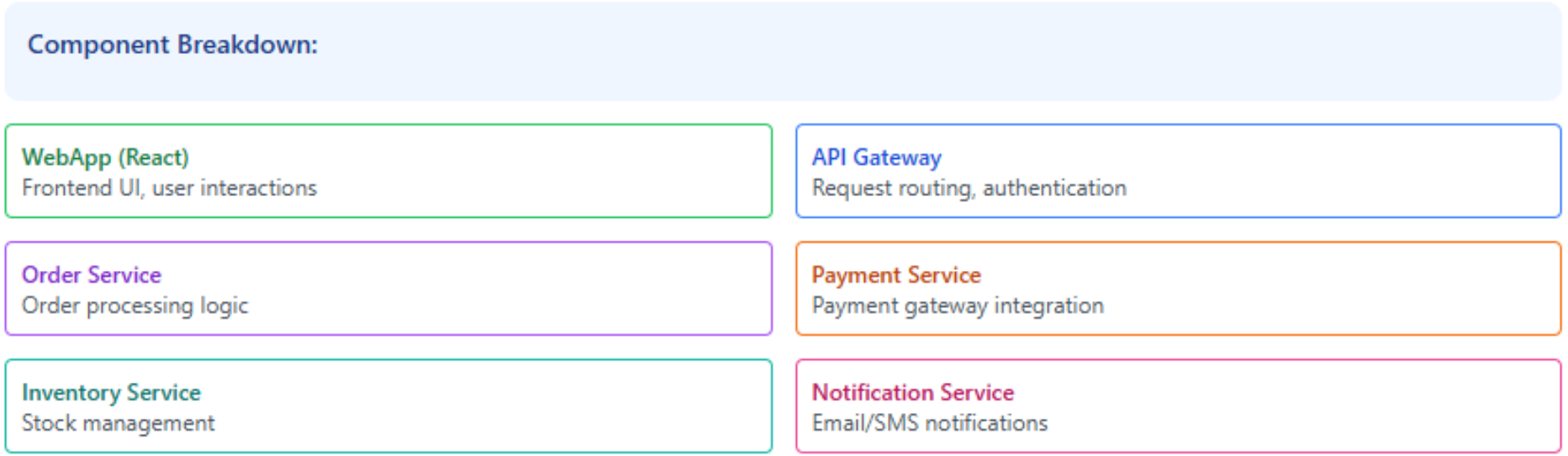
System Requirements:

- Users can browse products and place orders
- Real-time inventory management
- Payment processing integration
- Email notifications for orders
- Admin dashboard for management
- Mobile app support

We'll Design:

✓ Component diagram
✓ Deployment diagram
✓ Java code examples
✓ Configuration files

Case Study: Component Architecture



Key Interfaces:

- **IOrderManagement:** createOrder(), getOrderStatus()
- **IPaymentProcessor:** processPayment(), refund()
- **IInventoryCheck:** checkStock(), reserveItems()
- **INotificationSender:** sendEmail(), sendSMS()

Architecture Pattern: Microservices with API Gateway. Each service is independently deployable and scalable.

Code Example: Component Interfaces

```
// IOrderManagement.java - Provided Interface
public interface IOrderManagement {
    Order createOrder(OrderRequest request);
    Order getOrderStatus(String orderId);
    void cancelOrder(String orderId);
}

// IPaymentProcessor.java - Required Interface
public interface IPaymentProcessor {
    PaymentResult processPayment(PaymentRequest req);
    void refund(String transactionId);
}

// IInventoryCheck.java - Required Interface
public interface IInventoryCheck {
    boolean checkStock(String productId, int quantity);
    void reserveItems(List<OrderItem> items);
    void releaseItems(List<OrderItem> items);
}
```

Interfaces define the contracts between components:

Note: These interfaces enable loose coupling - we can swap implementations without affecting other components.

Code Example: Order Service Component

```
// OrderService.java - Component Implementation
@Service
@Component("OrderService")
public class OrderService implements IOrderManagement {

    // Required interfaces (dependencies)
    @Autowired
    private IPaymentProcessor paymentProcessor;

    @Autowired
    private IInventoryCheck inventoryService;

    @Autowired
    private INotificationSender notificationService;

    @Autowired
    private OrderRepository orderRepository;

    @Override
    public Order createOrder(OrderRequest request) {
        // 1. Validate inventory
        for (OrderItem item : request.getItems()) {
            if (!inventoryService.checkStock(
                item.getProductId(), item.getQuantity())) {
                throw new InsufficientStockException();
            }
        }
        // 2. Reserve inventory
        inventoryService.reserveItems(request.getItems());

        // 3. Process payment
        PaymentResult payment = paymentProcessor.processPayment(
            new PaymentRequest(request.getTotalAmount())
        );

        if (!payment.isSuccess()) {
            inventoryService.releaseItems(request.getItems());
            throw new PaymentFailedException();
        }

        // 4. Create order
        Order order = new Order(request, payment.getId());
        orderRepository.save(order);

        // 5. Send notification
        notificationService.sendEmail(
            request.getCustomerEmail(),
            "Order Confirmed: " + order.getId()
        );

        return order;
    }

    @Override
    public Order getOrderStatus(String orderId) {
        return orderRepository.findById(orderId)
            .orElseThrow(() -> new OrderNotFoundException());
    }
}
```

Best Practices and Design Principles

Component Design:

✓ **High Cohesion:** Keep related functionality together

✓ **Low Coupling:** Minimize dependencies between components

✓ **Interface Segregation:** Small, focused interfaces

✓ **Dependency Inversion:** Depend on abstractions, not concretions

✓ **Single Responsibility:** One reason to change

Deployment Design:

✓ **Scalability:** Design for horizontal scaling (add more nodes)

✓ **Redundancy:** No single points of failure

✓ **Security:** Network segmentation, firewalls, encryption

✓ **Monitoring:** Include logging and health check endpoints

✓ **Environment Parity:** Dev, staging, and prod should match

Common Pitfalls to Avoid:

✗ Too many fine-grained components (maintenance nightmare)

✗ Tight coupling between components (can't replace easily)

✗ Deploying everything on one server (no scalability)

✗ Ignoring network latency in distributed systems

✗ Hardcoding deployment configuration in code

Summary and Key takeaways

What We Learned:

Component Diagrams:
- Logical software structure
- Interfaces (provided/required)
- Ports and connectors
- Replaceable, modular design

Deployment Diagrams:
- Physical architecture
- Nodes (devices & execution envs)
- Artifacts and manifestation
- Communication paths

Summary and Key Takeaways

Remember: Good architecture is about making the right decisions at the right level. Component diagrams help you think about WHAT to build. Deployment diagrams help you think about WHERE and HOW to run it.

Why This Matters:

- Essential for system design and architecture roles
- Bridge between design and implementation
- Critical for DevOps and cloud deployment
- Enables team communication and documentation
- Foundation for microservices and distributed systems